Objects-VQA: How to Use It

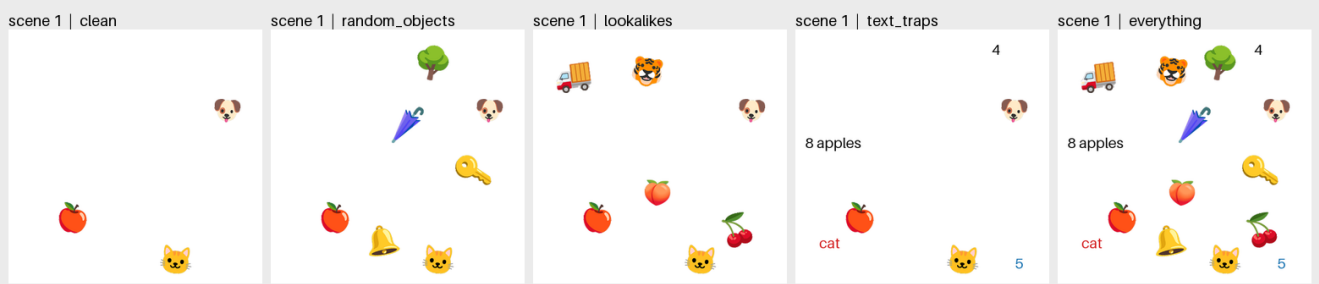
ATML project | huggingface.co/datasets/atml-shapes-vqa/objects-vqa | questions: ask Mohid

What it is

Our third dataset. Instead of shapes, the model counts everyday objects: **apples, cats, dogs and cars**. The distractors are close in looks *and* in meaning (a tomato is not an apple, a tiger is not a cat). **500 scenes x 5 versions = 2,500 images** (512 x 512).

Version	Diff.	What is added to the targets	What it tests
clean	0	nothing	baseline
random_objects	1	unrelated things: tree, key, guitar, gift ...	plain clutter
lookalikes	2	tomato / peach / cherries, tiger / lion / hamster, wolf / fox / bear, bus / truck / tractor	similar objects
text_traps	3	misleading text like "cat: 4", "3 dogs", "5" (always wrong)	being fooled by text
everything	4	all distractors from versions 1, 2 and 3 together	worst case

> **Key idea:** the target objects are pixel-identical in all 5 versions of a scene, so **all 5 versions have the same correct answer**. Each version adds one kind of distractor, so each version's accuracy drop tells you **which distractor** hurts the model most.



Every image uses the same question:

```
Count the objects in this image. Only count these four kinds of objects: apple, cat, dog, car.
Do not count anything else: similar-looking things (for example other fruit, other animals or
other vehicles) do not count, and neither do any words or numbers written in the image.
Answer in exactly this format: apple: <number>, cat: <number>, dog: <number>, car: <number>

answer:  apple: 2, cat: 1, dog: 0, car: 3        (all 4 classes, zeros included)
```

Versions 1-3 each add 2, 3 or 4 distractors; *everything* has exactly their union (6, 9 or 12). Splits are by scene: **train** 1,750 images (350 scenes), **validation** 375 (75), **test** 375 (75).

Useful columns

Column	Meaning
`image_id`	e.g. obj_0042_lookalikes. The key for your predictions
`scene_id`	same for all 5 versions of a scene. Use it to compare versions
`version`, `difficulty`	clean / random_objects / lookalikes / text_traps / everything = 0-4
`count_<object>`, `count_total`	true counts: count_apple, count_cat, count_dog, count_car
`objects`	JSON list of every object; for a lookalike, lookalike_of says which target it imitates

Step 1. Get access (skip if you already did this for the other datasets)

1. Join the **atml-shapes-vqa** organization on Hugging Face (Mohid sends the invite link). 2. At **huggingface.co/settings/tokens** create a token of type **Read**. 3. In a terminal:

```
pip install datasets huggingface_hub pillow
hf auth login          # paste your token (older versions: huggingface-cli login)
```

Step 2. Load the data (about 100 MB)

```
from datasets import load_dataset
ds = load_dataset("atml-shapes-vqa/objects-vqa")
test = ds["test"]            # 375 images = 75 scenes x 5 versions
ex = test[0]
ex["image"]       # PIL image, 512 x 512
ex["question"]    # the prompt (identical for every image)
ex["answer"]      # "apple: 2, cat: 1, dog: 0, car: 3"
ex["version"]     # clean / random_objects / lookalikes / text_traps / everything
look = test.filter(lambda e: e["version"] == "lookalikes")   # one version only
```

Step 3. Run your model and save its answers

Same as the other datasets: a CSV of `image_id`, `prediction`.

```
import csv
def ask_model(image, question):
    ...   # your VLM here (Qwen2-VL, LLaVA, InternVL, ...). Return its text answer.

with open("preds_objects_mymodel.csv", "w", newline="", encoding="utf-8") as f:
    w = csv.writer(f)
    w.writerow(["image_id", "prediction"])
    for ex in test:
        w.writerow([ex["image_id"], ask_model(ex["image"], ex["question"])])
```

Step 4. Score it

```
# once: download the scoring script (the same script works for all three datasets)
hf download atml-shapes-vqa/objects-vqa vqa_scoring.py --repo-type dataset --local-dir .

# score: it sees the "obj_" image ids and fetches the right answers by itself
python vqa_scoring.py --predictions preds_objects_mymodel.csv --split test
```

You get one row per version (exact match, accuracy per object, **overcnt** = how often the model counted distractors as real targets) and a **paired comparison against clean** with a McNemar p-value.

How to read the results

• Drop on `random_objects`: plain clutter is enough to hurt the model.

• Bigger drop on `lookalikes`: the model confuses similar objects. The per-object columns (and `lookalike_of` in `objects`) show which pairs, e.g. tomato vs apple.

• Drop on `text_traps`: it reads the text and copies its numbers.

• `everything`: do the distractors add up, or is one kind enough to break it?

Rules of thumb

• **Report results on the test split**; use train / validation for anything you tune. **Use the question as given**, so the 5 versions stay comparable. If something looks wrong, tell Mohid.

Object pictures: Noto Color Emoji by Google (SIL Open Font License 1.1).